AGI Jobs v0 (v2)

Demos Master Presentation

A production-grade command lattice for verifiable agentic work

What you are seeing

AGI Jobs v0 (v2) is engineered as a unified operational surface: agents + contracts + paymasters + orchestrators + simulations + demos — enforced by CI v2 so “main” is deployable truth.

Unified Intelligence Engine

One lattice composing mission-critical subsystems into a single control surface.

Deterministic Owner Control

Verified command center scripts keep pausing, upgrades, and parameters under owner authority.

Verifiable Operations

CI v2 produces auditable artefacts (status feeds, authority matrices, load sims) that gate merges.

Demos are rehearsal engines — not marketing assets

Each demo pressure-tests the control plane under conditions that matter: adversarial timing, parameter drift, and execution constraints.

What demos do (in practice)

- Stress economic parameters via Monte Carlo load simulations and compare dissipation metrics across configurations.
- Validate pause/unpause, upgrades, and rotation paths with deterministic owner command scripts.
- Produce artefacts (status feeds, matrices, plans) that travel with every decision: review → approval → deployment.
- Keep the “green wall” objective: no merge or release without reproducible verification.

Demo constellation map

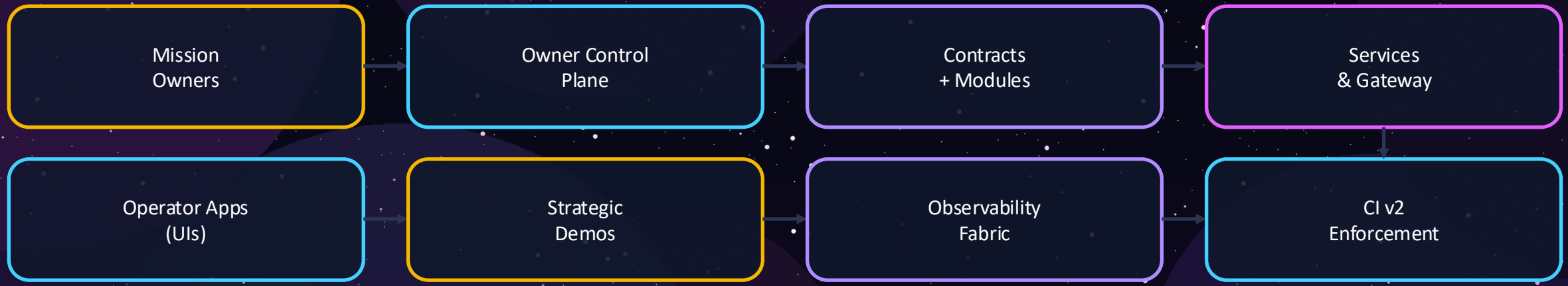
High-stakes rehearsals codified as reproducible scripts + UI bundles, enforced by companion workflows.

Tracks

- Sovereign Constellation — build-level rehearsal
- Celestial Archon — control doctrine
- Hypernova Governance — time-pressure tuning
- Kardashev II — omega-grade scenario envelopes

Architecture panorama

Owners command. Core executes. Observability proves. CI enforces.



Closed-loop control

Artefacts and CI outcomes cycle back to owners and operators as decision-grade proofs — a control loop that keeps promotion legible, bounded, and reversible.

Deterministic control plane

A pipeline that converts configuration reality into executable, auditable actions.

```
$ npm run owner:surface
$ npm run owner:parameters
$ npm run owner:doctor
$ npm run owner:mission-control
$ npm run owner:update-all
```

What the pipeline does

Surface scan fingerprints config drift → parameter matrices export a complete tuning map → doctor triage scores pass/warn/fail → mission-control condenses a decision brief → update-all executes deterministically and archives artefacts.

Why it matters

Run it in air-gapped control rooms. Every step emits files suitable for approvals, compliance, and post-mortems — and CI can refuse merges if owner control coverage is incomplete.

Pause / Resume as a first-class operation

When the lattice must stop, it stops deterministically — with previewed calldata and ownership checks.

```
$ npm run owner:system-pause -- --network <net>
# emits pause/unpause calldata + tx preview JSON
```

Operational contract

One command produces everything an operator needs: what will change, how it will be signed, and how it will be verified — before it touches the chain.

Recovery posture

Resume is treated as an audited change: regenerate authority matrices, re-check CI status feeds, and re-run the rehearsal that triggered the pause before restoring full throughput.

API-level verification (no local tooling)

Operators can corroborate the CI wall directly from the GitHub REST API with standard tools.

```
# latest workflow result on main
curl -s 'https://api.github.com/repos/MontrealAI/AGIJobsv0/actions/workflows/ci.yml/runs?branch=main&per_page=1' \
  | jq -r '.workflow_runs[0].status, .workflow_runs[0].conclusion'

# capture run id
RUN_ID=$(curl -s 'https://api.github.com/repos/MontrealAI/AGIJobsv0/actions/workflows/ci.yml/runs?branch=main&per_page=1' \
  | jq -r '.workflow_runs[0].id')

# enumerate job conclusions
curl -s "https://api.github.com/repos/MontrealAI/AGIJobsv0/actions/runs/${RUN_ID}/jobs?per_page=100" \
  | jq -r '.jobs[] | "\(.name): \(.conclusion)"'

# assert all success (exit code 0)
curl -s "https://api.github.com/repos/MontrealAI/AGIJobsv0/actions/runs/${RUN_ID}/jobs?per_page=100" \
  | jq -e 'all(.jobs[].conclusion == "success")'
```

Outcome — Independent parties can validate the same enforcement wall without cloning the repo: a portable proof primitive for institutions.

The green wall is policy, not preference

CI telemetry produces machine-readable feeds; branch protection scripts keep required contexts aligned with the workflow lattice.

Telemetry feed

reports/ci/status.json → dashboards
reports/ci/status.md → briefings

A local CLI reproduces the same wall on demand and gates deployments.

$ npm run ci:status-wall -- --require-success --include-companion --format json

Branch protection autopilot

ci:sync-contexts (check / regenerate)
ci:verify-contexts
ci:verify-companion-contexts
ci:verify-branch-protection
ci:enforce-branch-protection

Outcome: PR checks cannot go green unless all required contexts — including demo companions — pass.

Observability fabric: decision-grade artefacts

Everything important becomes a file: status, authority, plans, and stress-test traces.

Owner-control proofs

reports/owner-control/**
- authority matrices
- doctor reports
- command-center digest
- parameter plans

CI status feeds

reports/ci/status.{json,md}
- machine-readable wall
- briefing-ready mirror

Load simulations

reports/load-sim/**
- Monte Carlo CSV + JSON
- economic dissipation analysis

Sovereign Constellation Demo

Build-level rehearsal of the full constellation: core modules ↔ services ↔ operator surfaces ↔ proofs.

What it validates

- End-to-end integration between contracts, gateway/orchestrators, and apps
- Deterministic artefact production (reports/**)
- CI companion workflow gating (the demo must pass beside the main wall)

Signals to watch

Status feeds match the checks wall; demo outputs are reproducible locally and in CI.

Outcome

A demo that behaves like a deployment: if it fails, the lattice does not advance.

Celestial Archon Demonstration

A rehearsal of control doctrine: provenance, authority, and enforcement loops.

Core checks

- Allowed signers registry + provenance checks for release readiness
- Authority matrix regeneration and parity with enforced contexts
- CI guardrails report what is allowed to ship

How it feels in operation

Operators never “trust the vibe.” They trust signed artefacts, visible checks, and deterministic scripts that can be re-run by independent parties.

Hypernova Governance Demonstration

Governance under time pressure: recalibrate parameters without surrendering control.

What “governance” means here

A verifiable loop: owner council scripts regenerate the authority matrix; CI enforces parity with branch protection; artefacts cycle back into operator consoles and decision briefings.

Desired end state

Changes are legible, bounded, and reversible — without sacrificing pace.

Kardashev II • Omega-grade demo tracks

A suite of progressively harder rehearsal variants packaged as dedicated tracks in the repository.

Track roster (repo directories)

kardashev_ii_omega_grade_alpha_agi_business_3_demo
…_k2
…_k2_omega_upgrade
…_omega
…_ultra

kardashev_ii_omega_grade_upgrade_for_alpha_agi_business_3_demo
…_v2 / …_v3 / …_v4 / …_v5

Interpretation

Treat each track as a reproducible scenario envelope; compare artefacts and stability across envelopes before promoting changes.

Stability as a measurable property

The repository operationalises two cross-checkable ideas: simulation-based stress testing and thermostat-style control services.

Statistical physics lens (practical)

- Monte Carlo load sims generate traces
- Artefacts include “economic dissipation” analysis
- Owners re-tune parameters based on evidence, not intuition

Game-theoretic lens (practical)

Incentives and permissions are encoded as matrices and enforced contexts: if incentives drift, the wall blocks promotion.

Autopoiesis (operational)

Closed loops regenerate their own proofs: configs → matrices → actions → artefacts → CI gates → owners.

Quickstart for operators

Match the pinned toolchain so CI and local runs converge.

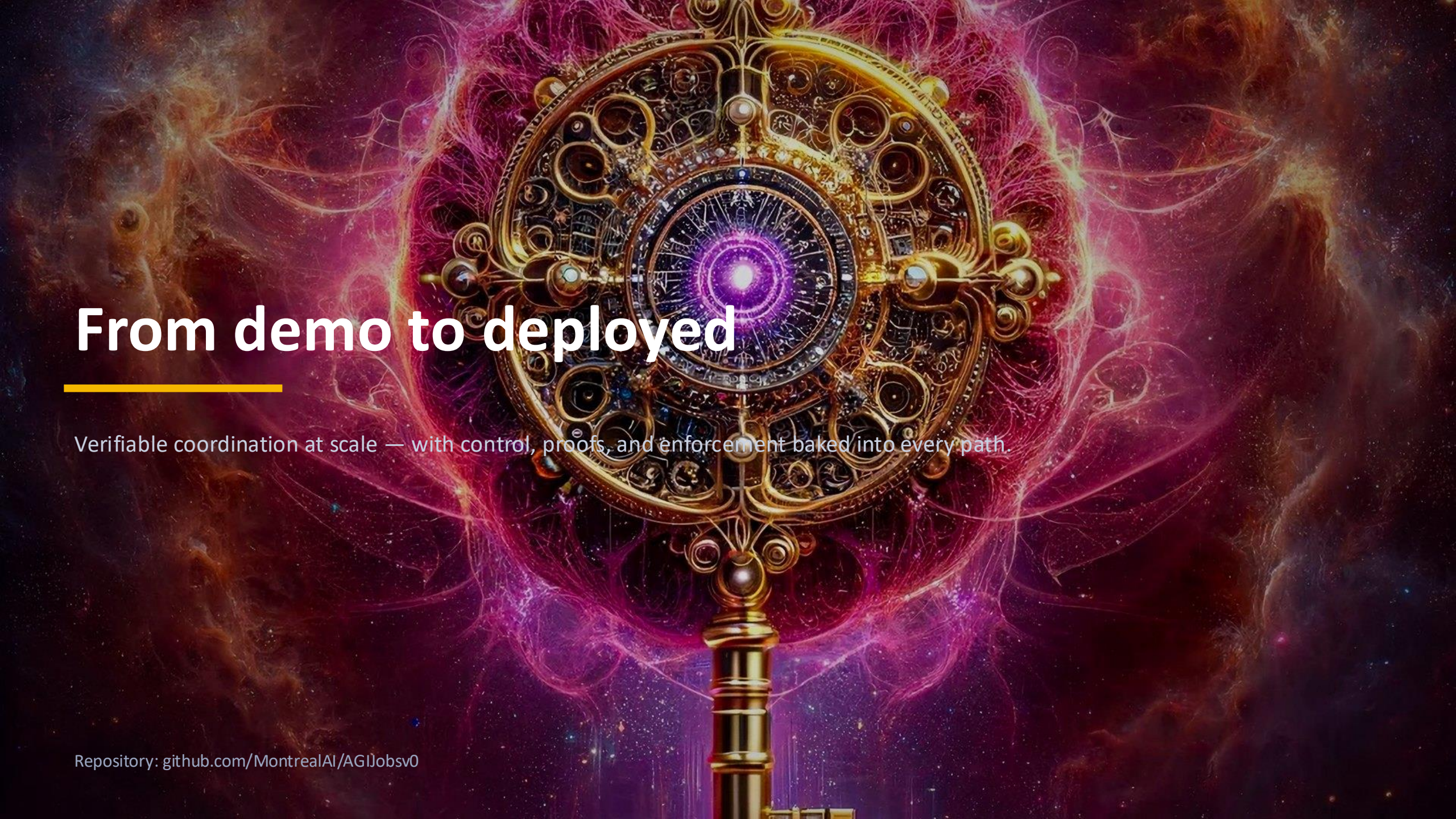
Toolchain

- Node.js 20.18.1 (pinned via .nvmrc / .node-version)
- Python 3.12
- Validate runtime: npm run doctor:node

- Principle: avoid “works on my machine.”

Promotion checklist

- Run owner:surface
- Export owner:parameters
- Resolve owner:doctor warnings
- Generate mission-control brief
- Produce safe bundle (plan)
- Execute update-all
- Verify via CI status feeds + API checks
- Re-run the target demo suite



From demo to deployed

Verifiable coordination at scale — with control, proofs, and enforcement baked into every path.

Repository: github.com/MontrealAI/AGIJobsv0